Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЕТ
к лабораторной работе №5
на тему:
«ЭЛЕМЕНТЫ СЕТЕВОГО ПРОГРАММИРОВАНИЯ»
БГУИР 6–05–0612–02 47

Выполнила студентка группы 453504
ЖУКОВСКАЯ Мария Александровна

(дата, подпись студента)

Проверил ст. преп. каф. Информатики
СЫС Анна Дмитриевна

(дата, подпись преподавателя)

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучить принципы организации сетевого взаимодействия процессов, получить практические навыки разработки клиентских и серверных приложений с использованием программного интерфейса сокетов операционной системы Windows.

2 ЗАДАНИЕ

Задание 1. Вариант 10. Удаленный интерпретатор операций с файлами. Протокол: TCP. Сервер работает только внутри специально выделенного учебного каталога sandbox.

Поддержать команды: LIST, SIZE filename, COUNT_LINES filename, READ_FIRST filename n, HELP.

Операции изменения и удаления произвольных файлов не предусматриваются.

Необходимо запретить выход за пределы каталога sandbox, включая относительные пути вида ../ и эквивалентные формы.

Эксперименты: один и три клиента; файлы разного размера.

Проверить: обработку отсутствующего файла, некорректного пути и одновременных запросов.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Архитектура разработанной программы

Система состоит из двух независимых приложений для Windows: сервера server.exe и клиента client.exe. Они запускаются как отдельные процессы и взаимодействуют только через TCP–соединение по адресу 127.0.0.1 и порту 20047. Сервер выполняет над файлами каталога sandbox только операции чтения по текстовым командам клиента: LIST, SIZE, COUNT_LINES, READ_FIRST, HELP.

Диаграмма архитектуры системы представлена на рисунке 3.1.

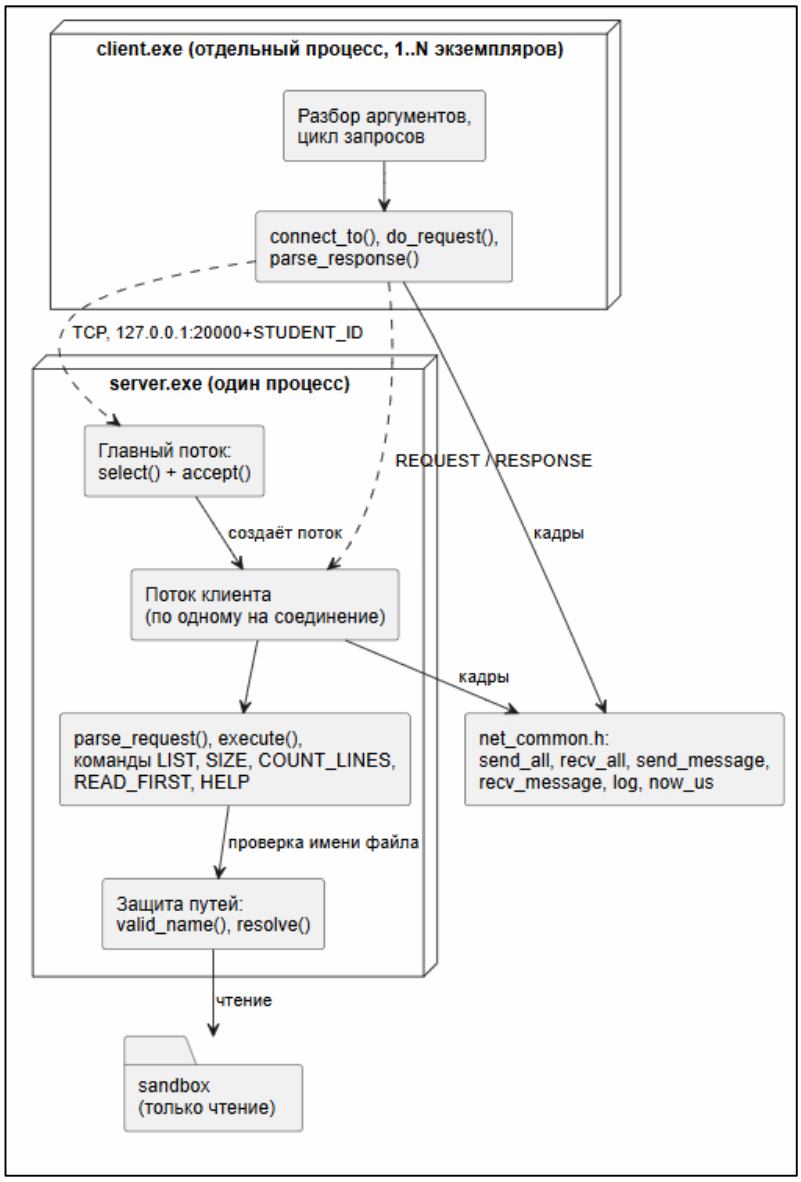


Рисунок 3.1 – Диаграмма архитектуры системы

Модули исходного кода представлены в таблице 3.1.

Таблица 3.1 – Модули исходного кода

Файл	Назначение
src/common/net_common.h	общий транспортный слой: отправка и прием данных, кадрирование сообщений, журнал, измерение времени, разбор аргументов
src/server/server.cpp	сервер: прием соединений, разбор запросов, выполнение команд, защита каталога sandbox
src/client/client.cpp	клиент: подключение, отправка запросов, разбор ответов, статистика, запись в CSV

Файл	Назначение
scripts/	сборка, генерация тестовых данных, запуск экспериментов, построение таблиц и графиков (Python)
tests/run_tests.py	автоматические проверки корректности

Между потоками сервера разделяются три объекта:

– флаг завершения g_stop: его выставляет обработчик Ctrl+C, читают главный поток и потоки клиентов;

– счетчик активных клиентов g_active: увеличивается главным потоком при accept, уменьшается потоком клиента при выходе;

– журнал: запись строки защищена мьютексом, чтобы сообщения разных потоков не перемешивались.

Остальные параметры задаются один раз до создания потоков и далее только читаются, поэтому дополнительной защиты не требуют.

Sandbox реализован как обычный каталог с проверками в коде сервера: любое имя файла, полученное от клиента, проверяется, и сервер не может выйти за пределы каталога. Это программное ограничение уровня приложения, а не изоляция средствами операционной системы.

3.2 Описание клиентской и серверной частей

Параметры запуска серверной части предсталены в таблице 3.2.

Таблица 3.2 – Параметры запуска серверной части

Параметр	По умолчанию	Назначение
--host	127.0.0.1	адрес, на котором сервер слушает
--port	20000	номер порта
--sandbox	sandbox	каталог с файлами; создается, если отсутствует
--timeout	30000	тайм–аут простоя клиента, мс
--delay	0	искусственная задержка ответа, мс (только для проверки тайм–аутов)

Сервер выполняет следующую последовательность дейтсвий:

1 Разбор параметров, проверка порта и тайм–аута, получение канонического пути sandbox.

2 Установка обработчика Ctrl+C, WSAStartup, создание сокета, bind, listen. При занятом порту выводится ошибка WSAEADDRINUSE, процесс завершается с кодом 1.

3 Главный цикл: select на слушающем сокете с тайм–аутом 0,5 с регулярно проверяет флаг завершения, при готовности выполняется accept,

затем происходят формирование адреса клиента, проверка лимита клиентов и создание потока клиента.

4 Поток клиента в цикле принимает запрос целиком, разбирает его, измеряет время выполнения команды и отправляет ответ. Цикл завершается при закрытии соединения клиентом, тайм–ауте простоя, ошибке, неверной длине кадра или запросе завершения сервера.

5 Штатное завершение по Ctrl+C потоки клиентов замечают не позже чем через 200 мс, закрывают сокеты, главный поток ожидает их завершения, закрывает слушающий сокет.

Реализованные команды к серверу представлены в таблице 3.3.

Таблица 3.3 – Команды к серверу

Команда	Реализация
LIST	перебор файлов sandbox, вывод пар «имя, размер», сортировка по имени
SIZE	размер файла в байтах
COUNT_LINES	чтение блоками по 64 КБ и подсчет символов перевода строки; последняя строка без \n учитывается; файл целиком в память не загружается
READ_FIRST	посимвольное чтение до n строк или до 60000 байт
HELP	справка по командам

Для защиты каталога sandbox была реализована двухэтапная проверка, выполняющая следующие действия

1. Проверка имени: допускаются только стандартные символы, длина до 100 символов, имя не может начинаться или заканчиваться точкой, содержать более двух точек и совпадать с именами устройств Windows (CON, NUL, COM1 и т. п.).

2. Проверка средствами файловой системы: отсутствие файла дает 404, символическая ссылка отклоняется, путь приводится к каноническому виду, и проверяется, что файл лежит непосредственно в sandbox и является обычным файлом.

Код реализованных проверок приведен в листинге 3.1.

Листинг 3.1 – Проверки безопасности путей

```
static bool valid_name(const std::string& n) {
    if (n.empty() || n.size() > 100) return false;
    for (unsigned char c : n)
        if (!(std::isalnum(c) || c == '.' || c == '_' || c == '-')) return
false;
    if (n.front() == '.' || n.back() == '.') return false;  // ".", "..",
скрытые
    if (n.find("..") != std::string::npos) return false;
    return !is_reserved(n);                                  // CON, NUL,
COM1...
}
```

```
    static bool resolve(const std::string& name, fs::path& out, Reply& e) {
        if (!valid_name(name)) { e = err_reply(403, "BAD_PATH invalid file
name"); return false; }
        std::error_code ec;
        fs::path p = g_root / name;
        fs::file_status st = fs::symlink_status(p, ec);
        if (ec || st.type() == fs::file_type::not_found) { e = err_reply(404,
"NOT_FOUND"); return false; }
        if (fs::is_symlink(st)) { e = err_reply(403, "BAD_PATH symlink not
allowed"); return false; }
        fs::path c = fs::canonical(p, ec);
        if (ec) { e = err_reply(404, "NOT_FOUND"); return false; }
        bool inside = fs::equivalent(c.parent_path(), g_root, ec);
        if (ec || !inside) { e = err_reply(403, "BAD_PATH outside sandbox");
return false; }
        if (!fs::is_regular_file(c, ec)) { e = err_reply(403, "BAD_PATH not
a regular file"); return false; }
        out = c;
        return true;
    }
```

Параметры запуска клиентской части представлены в таблице 3.4.

Таблица 3.4 – Параметры запуска клиентской части

Параметр	По умолчанию	Назначение
--host	127.0.0.1	адрес сервера
--port	20000	порт сервера
--id	1	идентификатор клиента
--timeout	5000	максимальное время ожидания
--count	1	число повторов команды —cmd
--cmd	не задан	команда для серии запросов
--csv, --label	не заданы	файл для записи итоговой строки статистики и имя конфигурации эксперимента
--quiet	выкл.	не печатать ответы

Клиент выполняет следующую последовательность действий:

1 Подключение с ограничением времени: сокет переводится в неблокирующий режим, connect возвращает WSAEWOULDBLOCK, затем select ожидает готовности соединения не дольше —timeout; затем сокет возвращается в блокирующий режим, включается TCP_NODELAY. Измеряется время установления соединения.

2 Для каждого запроса формируется сообщение V1 <requestId> <clientId> <команда>, засекается время, выполняется отправка и прием ответа целиком с тайм–аутом, разбирается заголовок ответа и проверяется совпадение номера запроса.

3 Ведется статистика числа отправленных и полученных запросов, успешных ответов, ошибок и тайм–аутов, переданные и принятые байты, суммы RTT и серверного времени.

4 После тайм–аута, обрыва соединения или нарушения протокола серия прекращается.

5 По завершении выводятся итоги.

3.3 Обоснование выбора TCP

Протокол TCP был выбран по следующим причинам, в отличие от UDP:

– полная и безошибочная доставка результата команды, так как TCP обеспечивает подтверждения и повторную передачу;

– TCP – поток байтов произвольной длины и его ответ достигает до 60000 байт, а на UDP потребовалось бы дробить на датаграммы и собирать с учётом перестановки.

– соответствие запросов и ответов по порядку;

– знание сервером о клиенте и его отключении;

– одновременная работа нескольких клиентов.

3.4 Описание сетевого протокола

Прикладной протокол работает поверх TCP. Клиент отправляет запрос и ожидает ответ на него, в рамках одного соединения запросы выполняются последовательно. Каждое сообщение передается в виде кадра с полем длины.

Сообщения идентифицируются двумя полями: requestId – сквозной номер запроса клиента, clientId – идентификатор участника.

В таблице 3.4 приведено описание протокола:

Таблица 3.5 – Описание протокола

Тип сообщения	Отправитель	Получатель	Содержимое	Ожидаемая реакция
Установление соединения (TCP, не сообщение протокола)	Client	Server	connect() на 127.0.0.1:20000+STUDENT_ID	сервер выполняет accept() и создает поток клиента
REQUEST	Client	Server	V1 <requestId> <clientId> <COMMAND> [args]	RESPONSE со статусом OK или ERR
RESPONSE (OK)	Server	Client	V1 <requestId> OK 200 <server_us> и тело результата	клиент проверяет requestId, фиксирует RTT, выводит результат

Тип сообщения	Отправитель	Получатель	Содержимое	Ожидаемая реакция
RESPONSE (ERR)	Server	Client	V1 <requestId> ERR <code> <server_us> и текст ошибки	клиент отображает ошибку; соединение остается открытым (кроме 413 и 503)
Завершение клиента (TCP, не сообщение протокола)	Client	Server	shutdown(SD_SEND), closesocket()	recv() на сервере возвращает 0, запись CLIENT_DISCONNECTED, поток завершается
Закрытие по инициативе сервера	Server	Client	закрытие сокета без сообщения: простой клиента дольше —timeout или завершение сервера (Ctrl+C)	клиент получает ошибку recv или признак закрытия соединения

Отдельных сообщений CONNECT и DISCONNECT нет: соединение открывается и закрывается средствами TCP, а идентификатор клиента передается в каждом запросе. Ответ один (RESPONSE) и несет статус OK или ERR, поэтому отдельные сообщения RESULT и ERROR не нужны.

Ниже представлен код листинга 3.2, содержащий отправку и принятие кадра.

Листинг 3.2 – Отправка и принятие кадра

```
inline bool send_message(SOCKET s, const std::string& payload) {
    uint32_t len = (uint32_t)payload.size();
    std::string frame;
    frame.resize(4 + payload.size());
    frame[0] = (char)((len >> 24) & 0xFF);
    frame[1] = (char)((len >> 16) & 0xFF);
    frame[2] = (char)((len >> 8) & 0xFF);
    frame[3] = (char)(len & 0xFF);
    std::memcpy(&frame[4], payload.data(), payload.size());
    return send_all(s, frame.data(), (int)frame.size());
}

    inline int recv_message(SOCKET s, std::string& out, int idle_ms, const
std::atomic<bool>* stop) {
        unsigned char h[4];
        int r = recv_all(s, (char*)h, 4, idle_ms, stop);
        if (r != RECV_OK) return r;
        uint32_t len = ((uint32_t)h[0] << 24) | ((uint32_t)h[1] << 16) |
((uint32_t)h[2] << 8) | (uint32_t)h[3];
        if (len == 0 || len > kMaxMessage) return RECV_BADLEN;
        out.resize(len);
```

```
    return recv_all(s, &out[0], (int)len, idle_ms, stop);
}
```

3.5 Структура передаваемых сообщений

TCP не сохраняет границы сообщений, поэтому каждое сообщение передается в виде кадра, содержимое которого приведено в таблице 3.6.

Таблица 3.6 – Содержимое кадра

Поле	Размер	Описание
Length	4 байта	беззнаковое 32–битное число в сетевом порядке байтов (старший байт первым); допустимо 1…65536
Payload	Length байт	текст сообщения

Максимальный размер одного сообщения – 65536 байт. Ответы на READ_FIRST и LIST ограничены 60000 байтами тела, поэтому с заголовком ответа укладываются в этот предел. Для больших данных сервер не разбивает ответ на несколько сообщений, а усекает его с пометкой [truncated].

Запрос имеет следующую структуру: V1 <requestId> <clientId> <COMMAND> [args...]. Он содержит версию протокола, номер запроса клиента, идентификатор клиента, команду, аргументы команды.

Ответ имеет следующую структуру: V1 <requestId> <OK|ERR> <code> <server_us>\n<тело>. Он содержит версию, номер запроса, статус, код и время выполнения команды на сервере. После символа перевода строки идет тело: результат для OK или текст ошибки для ERR.

Ниже приведен листинг 3.3, содержащий код формирования запроса.

Листинг 3.3 – Код формирования запроса

```
unsigned long long rid = next_id++;
std::string payload = "V1 " + std::to_string(rid) + " " + id + " " + command;
double t0 = now_us();
if (!send_message(s, payload)) { /* ошибка */ }
std::string msg;
int rr = recv_message(s, msg, timeout_ms, nullptr);
r.rtt_us = now_us() - t0;
```

Код разбора и нормализации запроса приведен в листинге 3.

Листинг 3. – Разбор и нормализация запроса

```
static Request parse_request(const std::string& s) {
    Request r;
    std::istringstream in(s);
    std::vector<std::string> t;
    std::string w;
    while (in >> w) {
        t.push_back(w);
        if (t.size() > 12) break;      // ограничение числа токенов
    }
    if (!t.empty() && t.size() >= 2) {
```

```
            unsigned long long id = 0;
            if (parse_uint(t[1], id)) r.id = id;
        }
        if (t.size() < 4 || t[0] != "V1" || r.id == 0) return r; // requestId
c 1
        r.client = t[2].substr(0, 16);
        r.cmd = t[3];
        for (auto& c : r.cmd) c = (char)std::toupper((unsigned char)c);
        r.args.assign(t.begin() + 4, t.end());
        r.valid = true;
        return r;
    }
```

3.6 Диаграмма последовательности сетевого взаимодействия

Диаграмма сетевого взаимодействия представлена на рисунке 3.2.

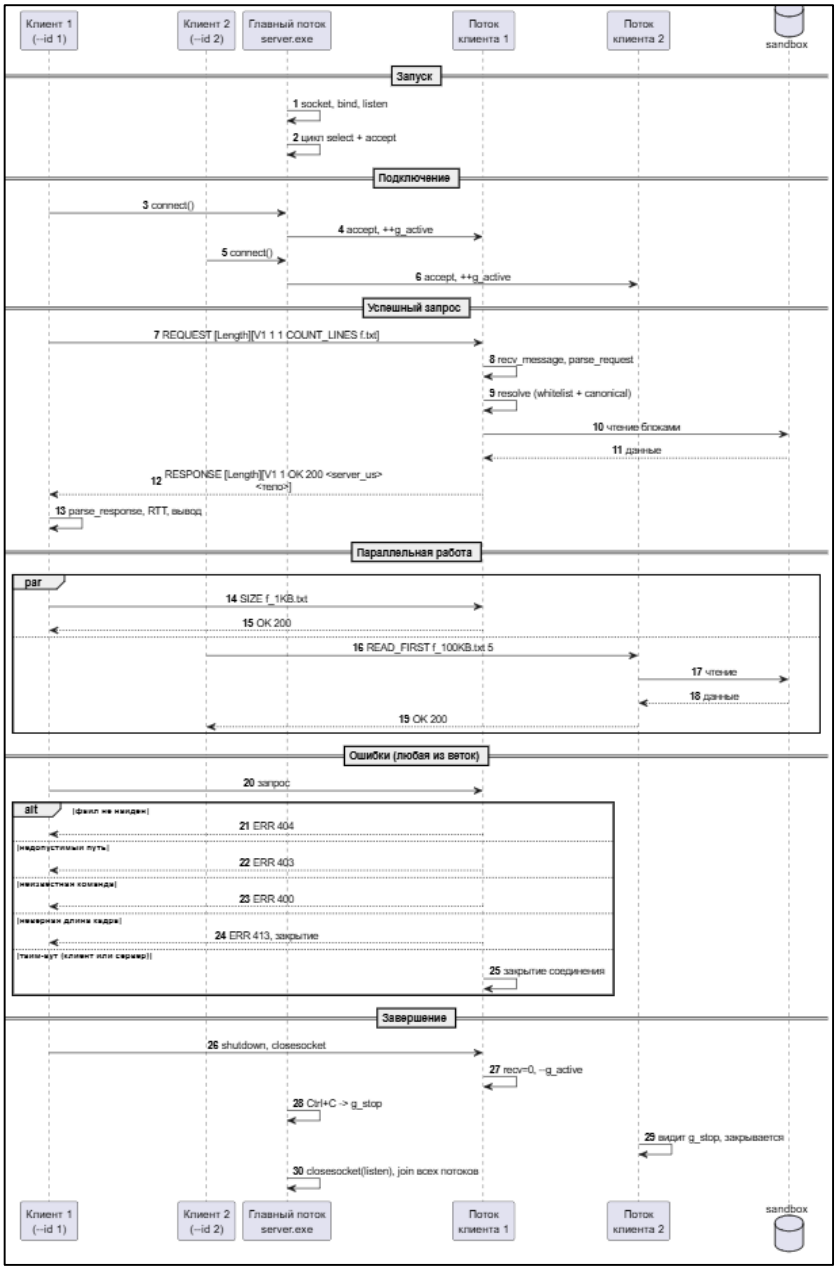


Рисунок 3.2 – Диаграмма сетевого взаимодействия

3.7 Перечень использованных функций Windows API

– WSAStartup(), WSACleanup() – инициализация и завершение Winsock;
– socket() – создание TCP–сокета;
– bind() – привязка сокета к адресу и порту;
– listen() – перевод сокета в режим ожидания соединений;
– accept() – приём входящего соединения, новый сокет клиента;
– connect() – установление соединения;
– send() – передача данных;
– recv() – приём данных;
– select() – ожидание готовности сокета с тайм–аутом;
– shutdown() – прекращение передачи (FIN) перед закрытием;
– closesocket() – закрытие сокета;
– WSAGetLastError() – код ошибки Winsock;
– ioctlsocket() – переключение сокета в неблокирующий режим и обратно;
– setsockopt() – TCP_NODELAY для отправки коротких сообщений без задержки;
– getsockopt() – SO_ERROR, причина неудачного подключения;
– inet_pton(), inet_ntop() – преобразование IP–адреса между текстом и двоичным видом;
– htons(), ntohs() – порядок байтов порта;
– FD_ZERO, FD_SET, FD_ISSET – работа с наборами сокетов для select;
– QueryPerformanceFrequency(), QueryPerformanceCounter() – высокоточное измерение времени;
– GetLocalTime() – метка времени в журнале;
– FormatMessageA(), LocalFree() – текстовое описание кода ошибки Windows;
– SetConsoleCtrlHandler() – обработка Ctrl+C для штатного завершения;
– Sleep() – искусственная задержка ответа.

3.8 Описание обработки ошибок и тайм–аутов

При ошибке любой функции Winsock приложение пишет в журнал имя функции, код WSAGetLastError() и текстовое описание, полученное через FormatMessageA: [18:40:02.117] CLIENT ERROR func=connect WSAGetLastError=10061 (<текст описания>).

Перечень обрабатываемых ситуаций представлен в таблице 3.7.

Таблица 3.7 – Обрабатываемые ситуации

Ситуация	Где обнаруживается	Реакция
Ошибка WSAStartup или создания сокета	сервер, клиент	сообщение с кодом, завершение процесса
Занятый порт (WSAEADDRINUSE, 10048)	bind на сервере	сообщение, закрытие сокета, WSACleanup, код возврата 1
Отказ соединения (WSAECONNREFUSED, 10061), сервер недоступен	connect в клиенте	сообщение SERVER_UNAVAILABLE, код возврата 2
Тайм-аут подключения	select в connect_to	CONNECT_TIMEOUT, код возврата 2
Тайм-аут ожидания ответа	recv_all в клиенте	TIMEOUT, соединение закрывается, код возврата 1
Тайм-аут простоя клиента	recv_all на сервере	TIMEOUT в журнале, соединение закрывается без сообщения
Закрытие соединения собеседником	recv вернул 0	CLIENT_DISCONNECTED (сервер), SERVER_CLOSED (клиент)
Сброс соединения (WSAECONNRESET, 10054)	recv, send	CLIENT_RESET, освобождение сокета и ресурсов
Ошибка отправки или приема	send_all, recv_all	код и описание в журнале, завершение обслуживания соединения
Частичная отправка и частичный прием	send_all, recv_all	цикл до полной передачи или приема нужного числа байтов
Некорректное сообщение	parse_request	ответ 400, соединение остается открытым
Недопустимая длина кадра	recv_message	ответ 413, соединение закрывается
Неизвестная команда	execute	ответ 400
Недопустимый путь, файл не найден	resolve	ответ 403 или 404

Ситуация	Где обнаруживается	Реакция
Исключение при обработке команды	блок try/catch в потоке клиента	ответ 500, сервер продолжает работу
Превышен лимит клиентов (64)	главный цикл сервера	ответ 503, соединение закрывается

По Ctrl+C сервер выставляет флаг завершения, потоки клиентов закрывают сокеты (shutdown, closesocket), главный поток ожидает их через join, закрывает слушающий сокет и вызывает WSACleanup. Клиент по окончании работы закрывает сокет и вызывает WSACleanup. Бесконечно работающих процессов после окончания экспериментов не остается.

3.10 Методики тестирования

Тестирование велось на трёх уровнях: автоматические проверки корректности, ручная демонстрация функциональности и эксперименты по производительности. Все проверки – на собственных сервисах через 127.0.0.1.

1 Автоматические тесты. Скрипт поднимает два сервера (обычный и с искусственной задержкой 1,5 с), вызывает клиента с разными командами, а для случаев, недоступных клиенту, использует «сырые» TCP-сокеты. Всего 34 проверки, охватывающие базовые команды и граничные случаи (COUNT_LINES на файле без завершающего \n и на пустом файле), отсутствующий файл, 13 форм некорректного пути (включая ../, ..\, UNC, C:\..., /etc/passwd, NTFS-потоки, CON, NUL, завершающую точку) с проверкой, что secret.txt вне sandbox не читается, неверные команды и аргументы, неверную длину кадра, частичную отправку, тайм-аут клиента, недоступный сервер и одновременную работу трёх клиентов по 50 запросов.

2 Ручная демонстрация. Сервер и клиенты в отдельных окнах. Показываются основные команды, отсутствующий файл, недопустимые пути, неверные аргументы, три клиента одновременно (чередование RECV id=1/2/3 в журнале), тайм-аут, недоступный сервер, занятый порт и штатное завершение по Ctrl+C.

3 Эксперименты. Тестовые файлы 1 КБ, 100 КБ, 10 МБ и 50 МБ из строк по 20–79 случайных латинских букв генерируются с фиксированным SEED. Матрица: число клиентов (1 и 3) × размер файла (4) × операция (SIZE,

COUNT_LINES, READ_FIRST) = 24 конфигурации, каждая трижды (72 запуска), по 20 запросов в каждом. Сервер один на весь эксперимент, клиенты параллельны, вывод ответов отключён (--quiet). Время через QueryPerformanceCounter разделяется на четыре интервала: подключение, RTT (сеть + сервер), время на сервере (только выполнение команды) и полное время клиентской операции. Адрес, порт, число запросов и клиентов задаются параметрами. Ограничения: всё на одном компьютере через петлевой интерфейс – сетевая задержка минимальна, ресурсы процессора делятся, кэш ОС ускоряет повторное чтение.

4 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ

4.1 Экспериментальные результаты

Результаты эксперимента приведены в таблице 4.1.

Таблица 4.1 – Результаты эксперимента

№	Конфигурация	Запуск 1, мс	Запуск 2, мс	Запуск 3, мс	Среднее, мс
1	1 клиент, 1 КБ, SIZE	0,225	0,198	0,209	0,211
2	1 клиент, 100 КБ, SIZE	0,188	0,199	0,203	0,197
3	1 клиент, 10 МБ, SIZE	0,181	0,191	0,196	0,189
4	1 клиент, 50 МБ, SIZE	0,197	0,200	0,199	0,199
5	3 клиента, 1 КБ, SIZE	0,214	0,236	0,231	0,227
6	3 клиента, 100 КБ, SIZE	0,248	0,239	0,223	0,237
7	3 клиента, 10 МБ, SIZE	0,284	0,315	0,271	0,290
8	3 клиента, 50 МБ, SIZE	0,248	0,269	0,258	0,258
9	1 клиент, 1 КБ, COUNT_LINES	0,588	0,278	0,273	0,380
10	1 клиент, 100 КБ, COUNT_LINES	0,645	0,284	0,283	0,404
11	1 клиент, 10 МБ, COUNT_LINES	8,562	7,222	7,555	7,780
12	1 клиент, 50 МБ, COUNT_LINES	36,256	33,218	35,664	35,046

13	3 клиента, 1 КБ, COUNT_LINES	0,281	0,299	0,292	0,291
14	3 клиента, 100 КБ, COUNT_LINES	0,424	0,378	0,392	0,398
15	3 клиента, 10 МБ, COUNT_LINES	13,230	12,101	12,535	12,622
16	3 клиента, 50 МБ, COUNT_LINES	65,974	67,824	71,254	68,351
17	1 клиент, 1 КБ, READ_FIRST	0,252	0,249	0,253	0,251
18	1 клиент, 100 КБ, READ_FIRST	0,348	0,356	0,349	0,351
19	1 клиент, 10 МБ, READ_FIRST	0,481	0,391	0,355	0,409
20	1 клиент, 50 МБ, READ_FIRST	0,349	0,346	0,366	0,354
21	3 клиента, 1 КБ, READ_FIRST	0,323	0,326	0,336	0,328
22	3 клиента, 100 КБ, READ_FIRST	0,482	0,515	0,520	0,506
23	3 клиента, 10 МБ, READ_FIRST	0,512	0,428	0,501	0,480
24	3 клиента, 50 МБ, READ_FIRST	0,401	0,524	0,504	0,477

Успешные запросы, суммы по всем запускам и клиентам представлены в таблице 4.2.

Таблица 4.2 – Успешные запросы и переданные данные

№	Конфигурация	Запросов отправлено	Успешных (OK)	Передано клиентом, байт	Получено клиентом, байт	Время подключения, мс	Время на сервере, мс
1	1 клиент, 1 КБ, SIZE	60	60	1533	1533	0,323	0,130

2	1 клиент, 100 КБ, SIZE	60	60	1653	1653	0,317	0,130
3	1 клиент, 10 МБ, SIZE	60	60	1593	1773	0,304	0,121
4	1 клиент, 50 МБ, SIZE	60	60	1593	1773	0,321	0,127
5	3 клиента, 1 КБ, SIZE	180	180	4599	4599	0,355	0,143
6	3 клиента, 100 КБ, SIZE	180	180	4959	4959	0,364	0,150
7	3 клиента, 10 МБ, SIZE	180	180	4779	5319	0,409	0,181
8	3 клиента, 50 МБ, SIZE	180	180	4779	5319	0,368	0,160
9	1 клиент, 1 КБ, COUNT_LINES	60	60	1953	1414	0,329	0,303
10	1 клиент, 100 КБ, COUNT_LINES	60	60	2073	1534	0,311	0,329
11	1 клиент, 10 МБ, COUNT_LINES	60	60	2013	1714	0,325	7,504
12	1 клиент, 50 МБ, COUNT_LINES	60	60	2013	1833	0,307	34,764
13	3 клиента, 1 КБ, COUNT_LINES	180	180	5859	4239	0,335	0,194
14	3 клиента, 100 КБ, COUNT_LINES	180	180	6219	4599	0,363	0,292
15	3 клиента, 10 МБ,	140	140	4697	4134	0,385	12,483

	COUNT_LINES						
16	3 клиента, 50 МБ, COUNT_LINES	160	160	5368	4890	0,404	68,379
17	1 клиент, 1 КБ, READ_FIRST	60	60	2133	65913	0,309	0,183
18	1 клиент, 100 КБ, READ_FIRST	60	60	2253	282573	0,315	0,261
19	1 клиент, 10 МБ, READ_FIRST	60	60	2193	309333	0,302	0,280
20	1 клиент, 50 МБ, READ_FIRST	60	60	2193	301653	0,298	0,263
21	3 клиента, 1 КБ, READ_FIRST	180	180	6399	197739	0,362	0,234
22	3 клиента, 100 КБ, READ_FIRST	180	180	6759	847719	0,408	0,379
23	3 клиента, 10 МБ, READ_FIRST	180	180	6579	928000	0,374	0,365
24	3 клиента, 50 МБ, READ_FIRST	180	180	6579	904959	0,374	0,362

Для примера был запущен сервер с тремя клиентами, один из которых также выполнил всевозможные команды. Окно сервера представлено на рис 4.1, окно выполняющего команды клиента представлено на рисунке 4.2.

```
PS C:\ddd\Unik\5sem\osisp\osisp_labs\lab05-network> build\server.exe --port 20010 --sandbox sandbox --timeout
600000
[20:43:01.299] SERVER SERVER START host=127.0.0.1 port=20010 sandbox=C:\ddd\Unik\5sem\osisp\osisp_labs\lab05-n
etwork\sandbox idle_timeout=600000ms delay=0ms
[20:43:10.627] SERVER CLIENT_CONNECTED peer=127.0.0.1:50451 conn=1 active=1
[20:43:24.273] SERVER CLIENT_CONNECTED peer=127.0.0.1:56342 conn=2 active=2
[20:43:28.273] SERVER CLIENT_CONNECTED peer=127.0.0.1:53752 conn=3 active=3
[20:43:39.823] SERVER RECV id=1 peer=127.0.0.1:50451 requestId=1 size=11 cmd=HELP
[20:43:39.823] SERVER SEND id=1 requestId=1 status=OK code=200 size=272 server_us=2
[20:43:47.065] SERVER RECV id=1 peer=127.0.0.1:50451 requestId=2 size=11 cmd=LIST
[20:43:47.065] SERVER SEND id=1 requestId=2 status=OK code=200 size=98 server_us=215
[20:44:03.867] SERVER RECV id=1 peer=127.0.0.1:50451 requestId=3 size=21 cmd=SIZE
[20:44:03.867] SERVER SEND id=1 requestId=3 status=OK code=200 size=5 server_us=381
[20:44:14.448] SERVER RECV id=1 peer=127.0.0.1:50451 requestId=4 size=29 cmd=COUNT_LINES
[20:44:14.464] SERVER SEND id=1 requestId=4 status=OK code=200 size=7 server_us=18090
[20:44:26.598] SERVER RECV id=1 peer=127.0.0.1:50451 requestId=5 size=31 cmd=READ_FIRST
[20:44:26.613] SERVER SEND id=1 requestId=5 status=OK code=200 size=223 server_us=13243
[20:44:44.953] SERVER CLIENT_DISCONNECTED id=1 peer=127.0.0.1:50451 conn=1
[20:45:53.682] SERVER CLIENT_RESET id=? peer=127.0.0.1:56342
[20:45:55.633] SERVER CLIENT_RESET id=? peer=127.0.0.1:53752
```

Рисунок 4.1 – Окно сервера

Вывод программы при запуске конфигурации Б представлен на рисунке 4.2.

```
PS C:\ddd\Unik\5sem\osisp\osisp_labs\lab05-network> build\client.exe --port 20010 --id 1
[20:43:10.627] CLIENT CONNECTED id=1 host=127.0.0.1 port=20010 connect=0.289ms
Type commands (HELP, LIST, SIZE f, COUNT_LINES f, READ_FIRST f n), QUIT to exit.
HELP
REQ 1 OK 200 rtt=0.699ms server=2us resp=290B
Commands:
  LIST                        list files in sandbox
  SIZE <file>                 file size in bytes
  COUNT_LINES <file>          number of lines
  READ_FIRST <file> <n>       first n lines (1..1000, at most 60000 bytes)
  HELP                        this text
LIST
REQ 2 OK 200 rtt=0.726ms server=215us resp=118B
empty.txt       0
f_100KB.txt     102426
f_10MB.txt      10485784
f_1KB.txt       1078
f_50MB.txt      52428818
noeol.txt       5
SIZE f_1KB.txt
REQ 3 OK 200 rtt=0.954ms server=381us resp=25B
1078
COUNT_LINES f_10MB.txt
REQ 4 OK 200 rtt=18.649ms server=18090us resp=29B
207466
READ_FIRST f_100KB.txt 5
REQ 5 OK 200 rtt=13.785ms server=13243us resp=245B
ihjcewuytlwicjkijlgerffim
nvebwvcduxcqrijpomjhhywupxvkepgvhescpkuqjpklikjdwrwavtpkyb
tufvqlikutizsnnjvlaziagguikptudhrpnxghioxgjufuekoxdxfxrrrfvsrtotagr
htiuxfpdpkuypjojyxcfbkysxvxjpal
tyuqgulqijipddaevlworshmkntrjusjjwckn
QUIT
[20:44:44.953] CLIENT SUMMARY id=1 sent=5 received=5 ok=5 err=0 timeouts=0 connect=0.289ms tota
l=94329.014ms avg_rtt=6.962ms avg_server=6.386ms bytes_out=123 bytes_in=707
```

Рисунок 4.2 – Окно клиента, выполняющего команды

4.2 Графики и анализ результатов

График зависимости времени выполнения от размера файла показан на рисунке 4.3.

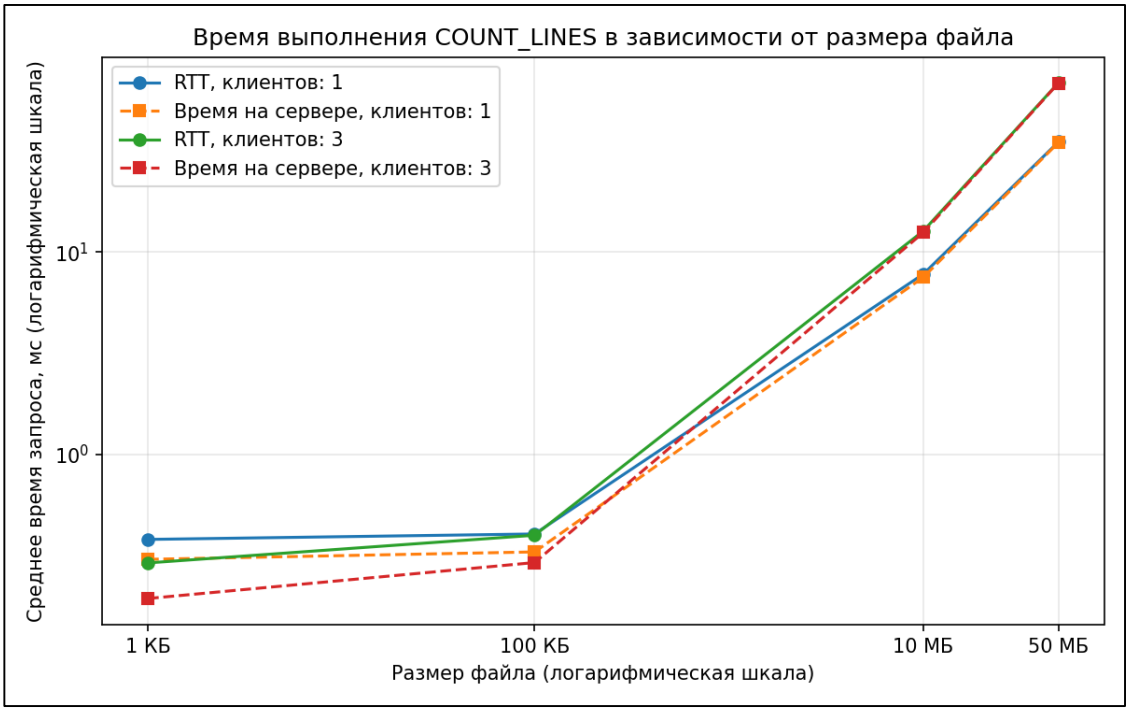


Рисунок 4.3 – График зависимости времени выполнения от размера файла

График пропускной способности приведен на рисунке 4.4.

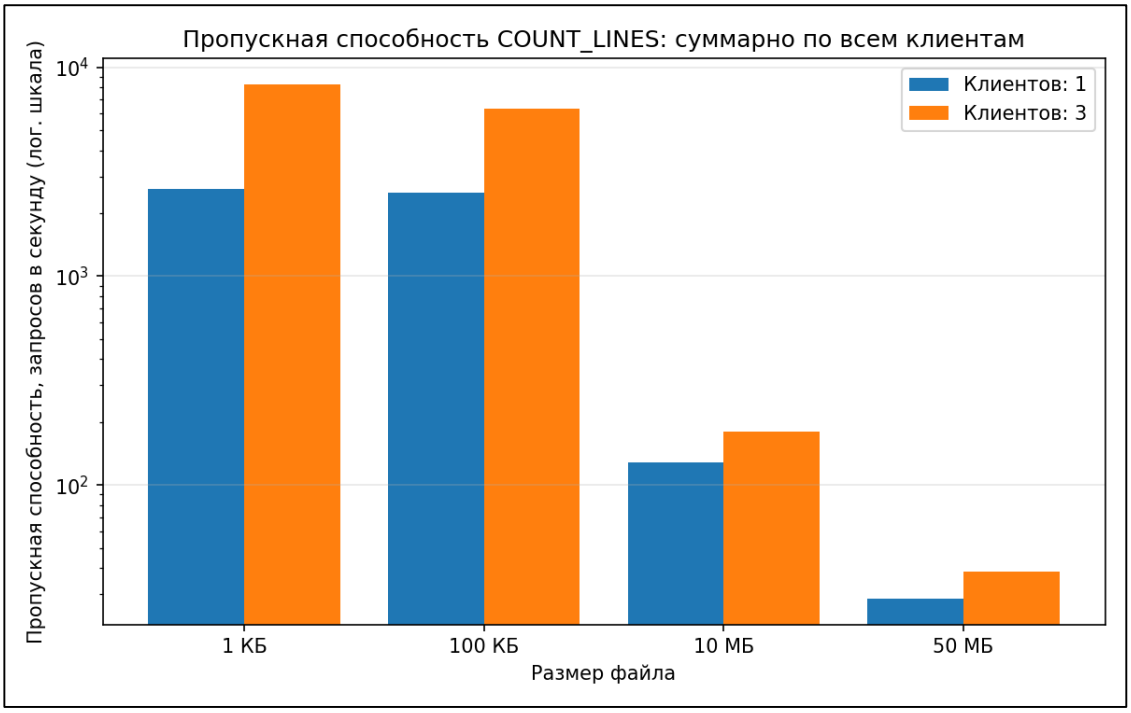


Рисунок 4.4 – График пропускной способности

Данные из таблиц 4.1 и 4.2 согласуются между собой:

– средний RTT из таблицы 2 для COUNT_LINES на 50 МБ при одном клиенте – 35,0 мс, и «время на сервере» из таблицы 4 – 34,8 мс: разница около 0,3 мс, что соответствует сетевым и прикладным накладным расходам;

– для 10 МБ: RTT 7,8 мс против 7,5 мс на сервере – та же картина;

– для 1 КБ и 100 КБ разница между RTT и временем на сервере больше относительно самих значений (например, 0,38 мс против 0,30 мс), что естественно: на малых временах накладные расходы составляют заметную долю.

Нулевые значения ошибок и тайм-аутов во всех 24 конфигурациях из таблицы 4.2 подтверждают устойчивость протокола и корректность обработки запросов при штатной работе.

Пропускная способность на рисунке 4.2 падает на несколько порядков по мере роста файла. Это ожидаемо: COUNT_LINES читает файл целиком, и время обработки растёт линейно с размером, а пропускная способность — обратно пропорционально.

На малых файлах (1 КБ, 100 КБ) три клиента дают кратный рост суммарной пропускной способности, то есть сервер успевает параллельно обрабатывать запросы разных клиентов. На 10 МБ и 50 МБ выигрыш от трёх клиентов снижается: узким местом становится процессорное время на чтение и подсчёт строк, и потоки начинают конкурировать за CPU.

На 10 МБ и 50 МБ число фактически отправленных запросов у трёх клиентов меньше 180 (140 и 160 по таблице 4.2), потому что часть запросов не успела выполниться за отведённое время эксперимента. Это дополнительно подтверждает, что тяжёлые операции ограничены вычислительной мощностью, а не протоколом.

На рисунке 4.1 кривые RTT и «время на сервере» почти совпадают на всём диапазоне, причём RTT немного выше. Это означает, что сетевая передача и накладные расходы клиента малы по сравнению с обработкой. При трёх клиентах обе кривые выше, чем при одном: на 10 МБ RTT растёт с 7,8 до 12,6 мс, на 50 МБ — с 35,0 до 68,4 мс. Это следствие конкуренции за процессор и дисковый ввод-вывод: три потока одновременно читают крупные файлы, и каждый запрос в среднем ждёт дольше.

ВЫВОДЫ

В ходе лабораторной работы была разработана клиент-серверная система удалённого интерпретатора файловых операций, работающая по TCP через интерфейс Winsock. Были изучены модели OSI и TCP/IP и освоены основные функции Windows Sockets. Реализован собственный протокол прикладного уровня с кадрированием сообщений, привязкой запросов к ответам по requestId и кодами статуса, что позволило корректно обрабатывать ошибки, разрывы соединения и тайм-ауты на обеих сторонах.

Сервер построен с ограничением числа клиентов и тайм-аутами, клиент поддерживает неблокирующее подключение с тайм-аутом и раздельный замер времени подключения, RTT и серверной обработки. Особое внимание уделено безопасности путей: двухуровневая проверка (whitelist-имя и канонизация с проверкой родительского каталога) надёжно отсекает ../, абсолютные и UNC-пути, симлинки и зарезервированные имена Windows, что подтверждено автоматическими тестами (34 проверки).

Эксперименты показали, что протокол устойчив и стабилен: во всех 24 конфигурациях не было ошибок и тайм-аутов. Время выполнения операций растёт линейно с размером файла, а накладные расходы сети на петлевом интерфейсе малы по сравнению с обработкой. На малых файлах три клиента дают почти линейный рост суммарной пропускной способности, на крупных — ограничение упирается в процессор. Таким образом, по выполнении данной лабораторной работы получены практические навыки разработки сетевых приложений, проектирования прикладного протокола и оценки корректности и производительности сетевого взаимодействия.

ИСПОЛЬЗОВАНИЕ ИСКУСТВЕННОГО ИНТЕЛЛЕКТА

Для всех случаев использовался Deepseek-V4.1-Flash.

Случай использования № 1.

Цель: Автоматический запуск приложения с файлами разного размера, с разным количеством клиентов.

Оригинальный промпт №1: Приведи примеры пайтон скриптов для автоматического запуска набора случаев проверяющих использование реализованного приложения для файлов различного размера.

Оригинальный промпт №2: В экспериментах должны проверяться случаи с одним, двумя, тремя клиентами для каждого файла.

Полученный результат после ручных изменений: Файлы gen_data.py, build.py, labutil.py, run_experiments.py.

Случай использования № 2.

Цель: Получение автоматической проверки базовых случаев работы приложения, а также проверка обработки приложением ошибок ввода, запуска и выхода за границы возможных ресурсов.

Оригинальный промпт №1: Помоги для моего приложения написать автоматические тесты на python для проверки базовых комманд.

Оригинальный промпт №2: Добавь в этот файл проверку некорректного пути и выхода за sandbox.

Оригинальный промпт №3: Дополни файл проверками на тайм-аут клиента.

Оригинальный промпт №4: Дополни файл проверками на одновременные запросы.

Полученный результат после ручных изменений: Фай run_tests.py.

Случай использования № 3.

Цель: Построение графиков.

Оригинальный промпт №1: Напиши файл для получения данных, с помощью которых можно построить графики зависимости времени выполнения от размера файла и пропускной способности.

Оригинальный промпт №2: На основе файла с результатами напиши пайтон файл для получения графика зависимости времени выполнения от размера файла.

Оригинальный промпт №3: Для этого же файла с результатами напиши файл для графика пропускной способности.

Полученный результат после ручных изменений: Файлы plotdata.py, plot_time.py, plot_throughput.py.

ПРИЛОЖЕНИЕ А. СВЕДЕНИЯ О РЕПОЗИТОРИИ GIT

Проект ведётся в Git. Репозиторий публичный:
– Платформа: GitHub
– URL: https://github.com/zhuchok0/osisp_labs.git
– Ветка по умолчанию: master
– Структура: osisp_labs/lab05-network/ (src, tests, data, scripts, diagrams, report)

Номера хэшов, содержащих основную реализацию программы представлены в таблице А.1.

Таблица А.1 – Хэши и соответствующее им содержимое

Хэш коммита	Описание
31f66d6	добавлена структура для пятой лабораторной работы
69e6120	добавлены общие функции обработки ошибок, времени, журналирования
77bd341	реализована функциональность сервера
a0b9a85	реализована функциональность клиента
23382e3	добавлены скрипты для генерации файлов, сборки сервера и клиента, запуска проекта для получения данных
d305848	добавлены тесты, файл README
f605f45	добавлен графический материал, а также файлы, использующиеся для его генерации

ПРИЛОЖЕНИЕ Б. ЛИСТИНГ КОДА ГЕНЕРАЦИИ ГРАФИЧЕСКИХ МАТЕРИАЛОВ

Для построения графиков используются файлы plot_time.py и fig_throughput.py, код которых приведен ниже.

Листинг файла plot_time.py:

```
import argparse
import sys
from collections import defaultdict
from statistics import mean

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from plotdata import ROOT, load_results, size_label

ap = argparse.ArgumentParser()
ap.add_argument("--csv", default=str(ROOT / "data" / "results.csv"))
ap.add_argument("--out", default=str(ROOT / "report" / "fig_time.png"))
ap.add_argument("--op",      default="COUNT_LINES",      choices=["SIZE",
"COUNT_LINES", "READ_FIRST"])
args = ap.parse_args()

rows = [r for r in load_results(args.csv) if r["op"] == args.op]
if not rows:
    sys.exit(f"В {args.csv} нет данных для операции {args.op}")

rtt, srv = defaultdict(list), defaultdict(list)   # (клиентов, размер) -
> значения по всем запускам и клиентам
for r in rows:
    rtt[(r["clients"], r["size"])].append(r["rtt_ms"])
    srv[(r["clients"], r["size"])].append(r["server_ms"])

sizes = sorted({k[1] for k in rtt})
fig, ax = plt.subplots(figsize=(8, 5))
for n in sorted({k[0] for k in rtt}):
    xs = [s for s in sizes if (n, s) in rtt]
    ax.plot(xs,  [mean(rtt[(n,  s)])  for  s  in  xs],  "o-",  label=f"RTT,
клиентов: {n}")
    ax.plot(xs, [mean(srv[(n, s)]) for s in xs], "s--", label=f"Время на
сервере, клиентов: {n}")

ax.set_xscale("log")
ax.set_yscale("log")
ax.set_xticks(sizes)
ax.set_xticklabels([size_label(s) for s in sizes])
ax.minorticks_off()
ax.set_title(f"Время  выполнения  {args.op}  в  зависимости  от  размера
файла")
ax.set_xlabel("Размер файла (логарифмическая шкала)")
ax.set_ylabel("Среднее время запроса, мс (логарифмическая шкала)")
ax.grid(True, which="both", alpha=0.3)
ax.legend()
fig.tight_layout()

fig.savefig(args.out, dpi=150)
print("Сохранено:", args.out)
```

Листинг файла fig_throughput.py:

```
import argparse
import sys
from collections import defaultdict
from statistics import mean

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from plotdata import ROOT, load_results, size_label

ap = argparse.ArgumentParser()
ap.add_argument("--csv", default=str(ROOT / "data" / "results.csv"))
ap.add_argument("--out",      default=str(ROOT      /      "report"      /
"fig_throughput.png"))
ap.add_argument("--op",      default="COUNT_LINES",      choices=["SIZE",
"COUNT_LINES", "READ_FIRST"])
ap.add_argument("--metric", default="rps", choices=["rps", "mbps"])
args = ap.parse_args()

rows = [r for r in load_results(args.csv) if r["op"] == args.op]
if not rows:
    sys.exit(f"В {args.csv} нет данных для операции {args.op}")

runs = defaultdict(list)  # (клиентов, размер, запуск) -> записи клиентов
этого запуска
for r in rows:
    runs[(r["clients"], r["size"], r["run"])].append(r)

value = defaultdict(list)  # (клиентов, размер) -> значения по запускам
for (n, size, _run), rs in runs.items():
    wall_s = max(x["total_ms"] for x in rs) / 1000.0
    rps = sum(x["received"] for x in rs) / wall_s
    value[(n, size)].append(rps if args.metric == "rps" else rps * size
/ 1024 ** 2)

sizes = sorted({k[1] for k in value})
clients = sorted({k[0] for k in value})
width = 0.8 / len(clients)
fig, ax = plt.subplots(figsize=(8, 5))
for i, n in enumerate(clients):
    xs  =  [j  +  (i  -  (len(clients)  -  1)  /  2)  *  width  for  j  in
range(len(sizes))]
    ys = [mean(value[(n, s)]) if (n, s) in value else 0 for s in sizes]
    ax.bar(xs, ys, width, label=f"Клиентов: {n}")

ax.set_xticks(range(len(sizes)))
ax.set_xticklabels([size_label(s) for s in sizes])
ax.set_yscale("log")
unit = "запросов в секунду" if args.metric == "rps" else "МБ/с"
ax.set_title(f"Пропускная  способность  {args.op}:  суммарно  по  всем
клиентам")
ax.set_xlabel("Размер файла")
ax.set_ylabel(f"Пропускная способность, {unit} (лог. шкала)")
ax.grid(True, axis="y", alpha=0.3)
ax.legend()
fig.tight_layout()
fig.savefig(args.out, dpi=150)
print("Сохранено:", args.out)
```

ПРИЛОЖЕНИЕ В. ЛИСТИНГ ФАЙЛОВ ДЛЯ ПОСТРОЕНИЯ ДИАГРАММ

Ниже представлены файлы для построения диаграмм architecture.puml и sequence.puml.

Листинг файла architecture.puml:

```
@startuml architecture
!pragma layout smetana
title Вариант 10. Архитектура приложения

skinparam shadowing false
skinparam componentStyle rectangle

node "client.exe (отдельный процесс, 1..N экземпляров)" as CLI {
    [Разбор аргументов,\nцикл запросов] as CMAIN
    [connect_to(), do_request(),\nparse_response()] as CREQ
}

node "server.exe (один процесс)" as SRV {
    [Главный поток:\nselect() + accept()] as SMAIN
    [Поток клиента\n(по одному на соединение)] as STH
    [parse_request(),       execute(),\nкоманды       LIST,       SIZE,
COUNT_LINES,\nREAD_FIRST, HELP] as SEXE
    [Защита путей:\nvalid_name(), resolve()] as SSEC
}

component               "net_common.h:\nsend_all,               recv_all,
send_message,\nrecv_message, log, now_us" as NC

folder "sandbox\n(только чтение)" as SB

CMAIN --> CREQ
CREQ --> NC : кадры
SMAIN --> STH : создаёт поток
STH --> SEXE
SEXE --> SSEC : проверка имени файла
SSEC --> SB : чтение
STH --> NC : кадры
CREQ ..> SMAIN : TCP, 127.0.0.1:20000+STUDENT_ID
CREQ ..> STH : REQUEST / RESPONSE
@enduml
```

Листинг файла sequence.puml:

```
@startuml sequence
title Вариант 10. Удалённый интерпретатор файловых операций (TCP, протокол
V1)

skinparam shadowing false
skinparam sequenceMessageAlign center
autonumber

participant "Клиент 1\n(--id 1)" as C1
participant "Клиент 2\n(--id 2)" as C2
participant "Главный поток\nserver.exe" as M
participant "Поток\nклиента 1" as T1
participant "Поток\nклиента 2" as T2
database "sandbox" as FS
```

```
== Запуск ==
M -> M : socket, bind, listen
M -> M : цикл select + accept

== Подключение ==
C1 -> M : connect()
M -> T1 : accept, ++g_active
C2 -> M : connect()
M -> T2 : accept, ++g_active

== Успешный запрос ==
C1 -> T1 : REQUEST [Length][V1 1 1 COUNT_LINES f.txt]
T1 -> T1 : recv_message, parse_request
T1 -> T1 : resolve (whitelist + canonical)
T1 -> FS : чтение блоками
FS --> T1 : данные
T1 --> C1 : RESPONSE [Length][V1 1 OK 200 <server_us>\n<тело>]
C1 -> C1 : parse_response, RTT, вывод

== Параллельная работа ==
par
    C1 -> T1 : SIZE f_1KB.txt
    T1 --> C1 : OK 200
else
    C2 -> T2 : READ_FIRST f_100KB.txt 5
    T2 -> FS : чтение
    FS --> T2 : данные
    T2 --> C2 : OK 200
end

== Ошибки (любая из веток) ==
C1 -> T1 : запрос
alt файл не найден
    T1 --> C1 : ERR 404
else недопустимый путь
    T1 --> C1 : ERR 403
else неизвестная команда
    T1 --> C1 : ERR 400
else неверная длина кадра
    T1 --> C1 : ERR 413, закрытие
else тайм-аут (клиент или сервер)
    T1 -> T1 : закрытие соединения
end

== Завершение ==
C1 -> T1 : shutdown, closesocket
T1 -> T1 : recv=0, --g_active
M -> M : Ctrl+C -> g_stop
T2 -> T2 : видит g_stop, закрывается
M -> M : closesocket(listen), join всех потоков
@enduml
```

ПРИЛОЖЕНИЕ Г. ИСХОДНЫЕ ЭКСПЕРИМЕНТАЛЬНЫЕ ДАННЫЕ

Экспериментальные данные представлены в файле results.csv:

```
      label,client_id,cmd,sent,received,ok,err,timeouts,connect_ms,total_ms,av
g_rtt_ms,avg_server_ms,bytes_out,bytes_in
      c1_f_1KB.txt_SIZE_r1,1,"SIZE
f_1KB.txt",20,20,20,0,0,0.307,5.359,0.225,0.126,511,511
      c1_f_1KB.txt_SIZE_r2,1,"SIZE
f_1KB.txt",20,20,20,0,0,0.352,4.887,0.198,0.129,511,511
      c1_f_1KB.txt_SIZE_r3,1,"SIZE
f_1KB.txt",20,20,20,0,0,0.311,5.025,0.209,0.136,511,511
      c1_f_1KB.txt_COUNT_LINES_r1,1,"COUNT_LINES
f_1KB.txt",20,20,20,0,0,0.325,12.627,0.588,0.516,651,472
      c1_f_1KB.txt_COUNT_LINES_r2,1,"COUNT_LINES
f_1KB.txt",20,20,20,0,0,0.333,6.454,0.278,0.196,651,471
      c1_f_1KB.txt_COUNT_LINES_r3,1,"COUNT_LINES
f_1KB.txt",20,20,20,0,0,0.330,6.375,0.273,0.196,651,471
      c1_f_1KB.txt_READ_FIRST_r1,1,"READ_FIRST                          f_1KB.txt
100",20,20,20,0,0,0.314,5.890,0.252,0.184,711,21971
      c1_f_1KB.txt_READ_FIRST_r2,1,"READ_FIRST                          f_1KB.txt
100",20,20,20,0,0,0.315,5.887,0.249,0.181,711,21971
      c1_f_1KB.txt_READ_FIRST_r3,1,"READ_FIRST                          f_1KB.txt
100",20,20,20,0,0,0.298,5.900,0.253,0.184,711,21971
      c1_f_100KB.txt_SIZE_r1,1,"SIZE
f_100KB.txt",20,20,20,0,0,0.357,4.615,0.188,0.126,551,551
      c1_f_100KB.txt_SIZE_r2,1,"SIZE
f_100KB.txt",20,20,20,0,0,0.292,4.763,0.199,0.129,551,551
      c1_f_100KB.txt_SIZE_r3,1,"SIZE
f_100KB.txt",20,20,20,0,0,0.302,4.857,0.203,0.134,551,551
      c1_f_100KB.txt_COUNT_LINES_r1,1,"COUNT_LINES
f_100KB.txt",20,20,20,0,0,0.298,13.741,0.645,0.563,691,512
      c1_f_100KB.txt_COUNT_LINES_r2,1,"COUNT_LINES
f_100KB.txt",20,20,20,0,0,0.328,6.612,0.284,0.215,691,511
      c1_f_100KB.txt_COUNT_LINES_r3,1,"COUNT_LINES
f_100KB.txt",20,20,20,0,0,0.306,6.492,0.283,0.210,691,511
      c1_f_100KB.txt_READ_FIRST_r1,1,"READ_FIRST                      f_100KB.txt
100",20,20,20,0,0,0.299,7.812,0.348,0.260,751,94191
      c1_f_100KB.txt_READ_FIRST_r2,1,"READ_FIRST                      f_100KB.txt
100",20,20,20,0,0,0.343,8.143,0.356,0.263,751,94191
      c1_f_100KB.txt_READ_FIRST_r3,1,"READ_FIRST                      f_100KB.txt
100",20,20,20,0,0,0.302,7.840,0.349,0.260,751,94191
      c1_f_10MB.txt_SIZE_r1,1,"SIZE
f_10MB.txt",20,20,20,0,0,0.304,4.432,0.181,0.115,531,591
      c1_f_10MB.txt_SIZE_r2,1,"SIZE
f_10MB.txt",20,20,20,0,0,0.303,4.613,0.191,0.124,531,591
      c1_f_10MB.txt_SIZE_r3,1,"SIZE
f_10MB.txt",20,20,20,0,0,0.305,4.756,0.196,0.123,531,591
      c1_f_10MB.txt_COUNT_LINES_r1,1,"COUNT_LINES
f_10MB.txt",20,20,20,0,0,0.302,172.296,8.562,8.315,671,572
      c1_f_10MB.txt_COUNT_LINES_r2,1,"COUNT_LINES
f_10MB.txt",20,20,20,0,0,0.353,145.564,7.222,6.915,671,571
      c1_f_10MB.txt_COUNT_LINES_r3,1,"COUNT_LINES
f_10MB.txt",20,20,20,0,0,0.321,152.165,7.555,7.282,671,571
      c1_f_10MB.txt_READ_FIRST_r1,1,"READ_FIRST                       f_10MB.txt
100",20,20,20,0,0,0.279,10.474,0.481,0.302,731,103111
      c1_f_10MB.txt_READ_FIRST_r2,1,"READ_FIRST                       f_10MB.txt
100",20,20,20,0,0,0.335,8.704,0.391,0.277,731,103111
      c1_f_10MB.txt_READ_FIRST_r3,1,"READ_FIRST                       f_10MB.txt
100",20,20,20,0,0,0.293,7.932,0.355,0.261,731,103111
```

```
      c1_f_50MB.txt_SIZE_r1,1,"SIZE
f_50MB.txt",20,20,20,0,0,0.353,4.889,0.197,0.126,531,591
      c1_f_50MB.txt_SIZE_r2,1,"SIZE
f_50MB.txt",20,20,20,0,0,0.293,4.784,0.200,0.127,531,591
      c1_f_50MB.txt_SIZE_r3,1,"SIZE
f_50MB.txt",20,20,20,0,0,0.317,4.805,0.199,0.128,531,591
      c1_f_50MB.txt_COUNT_LINES_r1,1,"COUNT_LINES
f_50MB.txt",20,20,20,0,0,0.305,726.143,36.256,35.963,671,611
      c1_f_50MB.txt_COUNT_LINES_r2,1,"COUNT_LINES
f_50MB.txt",20,20,20,0,0,0.305,665.348,33.218,32.952,671,611
      c1_f_50MB.txt_COUNT_LINES_r3,1,"COUNT_LINES
f_50MB.txt",20,20,20,0,0,0.310,714.390,35.664,35.376,671,611
      c1_f_50MB.txt_READ_FIRST_r1,1,"READ_FIRST                    f_50MB.txt
100",20,20,20,0,0,0.296,7.817,0.349,0.259,731,100551
      c1_f_50MB.txt_READ_FIRST_r2,1,"READ_FIRST                    f_50MB.txt
100",20,20,20,0,0,0.298,7.767,0.346,0.258,731,100551
      c1_f_50MB.txt_READ_FIRST_r3,1,"READ_FIRST                    f_50MB.txt
100",20,20,20,0,0,0.300,8.169,0.366,0.272,731,100551
      c3_f_1KB.txt_SIZE_r1,1,"SIZE
f_1KB.txt",20,20,20,0,0,0.431,5.705,0.232,0.144,511,511
      c3_f_1KB.txt_SIZE_r1,2,"SIZE
f_1KB.txt",20,20,20,0,0,0.337,5.427,0.227,0.145,511,511
      c3_f_1KB.txt_SIZE_r1,3,"SIZE
f_1KB.txt",20,20,20,0,0,0.295,4.465,0.182,0.117,511,511
      c3_f_1KB.txt_SIZE_r2,1,"SIZE
f_1KB.txt",20,20,20,0,0,0.380,5.699,0.236,0.149,511,511
      c3_f_1KB.txt_SIZE_r2,2,"SIZE
f_1KB.txt",20,20,20,0,0,0.321,5.978,0.250,0.158,511,511
      c3_f_1KB.txt_SIZE_r2,3,"SIZE
f_1KB.txt",20,20,20,0,0,0.321,5.360,0.223,0.136,511,511
      c3_f_1KB.txt_SIZE_r3,1,"SIZE
f_1KB.txt",20,20,20,0,0,0.347,5.900,0.234,0.146,511,511
      c3_f_1KB.txt_SIZE_r3,2,"SIZE
f_1KB.txt",20,20,20,0,0,0.449,6.043,0.240,0.150,511,511
      c3_f_1KB.txt_SIZE_r3,3,"SIZE
f_1KB.txt",20,20,20,0,0,0.318,5.302,0.219,0.139,511,511
      c3_f_1KB.txt_COUNT_LINES_r1,1,"COUNT_LINES
f_1KB.txt",20,20,20,0,0,0.337,6.777,0.290,0.199,651,471
      c3_f_1KB.txt_COUNT_LINES_r1,2,"COUNT_LINES
f_1KB.txt",20,20,20,0,0,0.306,6.713,0.291,0.199,651,471
      c3_f_1KB.txt_COUNT_LINES_r1,3,"COUNT_LINES
f_1KB.txt",20,20,20,0,0,0.304,6.112,0.262,0.180,651,471
      c3_f_1KB.txt_COUNT_LINES_r2,1,"COUNT_LINES
f_1KB.txt",20,20,20,0,0,0.355,7.002,0.296,0.200,651,471
      c3_f_1KB.txt_COUNT_LINES_r2,2,"COUNT_LINES
f_1KB.txt",20,20,20,0,0,0.342,7.458,0.325,0.222,651,471
      c3_f_1KB.txt_COUNT_LINES_r2,3,"COUNT_LINES
f_1KB.txt",20,20,20,0,0,0.319,6.458,0.276,0.190,651,471
      c3_f_1KB.txt_COUNT_LINES_r3,1,"COUNT_LINES
f_1KB.txt",20,20,20,0,0,0.402,7.008,0.297,0.195,651,471
      c3_f_1KB.txt_COUNT_LINES_r3,2,"COUNT_LINES
f_1KB.txt",20,20,20,0,0,0.309,7.555,0.330,0.190,651,471
      c3_f_1KB.txt_COUNT_LINES_r3,3,"COUNT_LINES
f_1KB.txt",20,20,20,0,0,0.339,5.907,0.248,0.169,651,471
      c3_f_1KB.txt_READ_FIRST_r1,1,"READ_FIRST                     f_1KB.txt
100",20,20,20,0,0,0.374,8.130,0.351,0.246,711,21971
      c3_f_1KB.txt_READ_FIRST_r1,2,"READ_FIRST                     f_1KB.txt
100",20,20,20,0,0,0.381,7.443,0.318,0.227,711,21971
      c3_f_1KB.txt_READ_FIRST_r1,3,"READ_FIRST                     f_1KB.txt
100",20,20,20,0,0,0.356,6.925,0.299,0.213,711,21971
      c3_f_1KB.txt_READ_FIRST_r2,1,"READ_FIRST                     f_1KB.txt
100",20,20,20,0,0,0.382,8.396,0.366,0.254,711,21971
      c3_f_1KB.txt_READ_FIRST_r2,2,"READ_FIRST                     f_1KB.txt
100",20,20,20,0,0,0.359,7.639,0.329,0.243,711,21971
```

```
      c3_f_1KB.txt_READ_FIRST_r2,3,"READ_FIRST                    f_1KB.txt
100",20,20,20,0,0,0.334,6.593,0.282,0.202,711,21971
      c3_f_1KB.txt_READ_FIRST_r3,1,"READ_FIRST                    f_1KB.txt
100",20,20,20,0,0,0.344,8.204,0.357,0.255,711,21971
      c3_f_1KB.txt_READ_FIRST_r3,2,"READ_FIRST                    f_1KB.txt
100",20,20,20,0,0,0.408,7.891,0.339,0.243,711,21971
      c3_f_1KB.txt_READ_FIRST_r3,3,"READ_FIRST                    f_1KB.txt
100",20,20,20,0,0,0.324,7.196,0.313,0.226,711,21971
      c3_f_100KB.txt_SIZE_r1,1,"SIZE
f_100KB.txt",20,20,20,0,0,0.363,6.444,0.271,0.170,551,551
      c3_f_100KB.txt_SIZE_r1,2,"SIZE
f_100KB.txt",20,20,20,0,0,0.354,5.863,0.241,0.152,551,551
      c3_f_100KB.txt_SIZE_r1,3,"SIZE
f_100KB.txt",20,20,20,0,0,0.344,5.598,0.232,0.147,551,551
      c3_f_100KB.txt_SIZE_r2,1,"SIZE
f_100KB.txt",20,20,20,0,0,0.382,5.919,0.244,0.151,551,551
      c3_f_100KB.txt_SIZE_r2,2,"SIZE
f_100KB.txt",20,20,20,0,0,0.446,6.074,0.250,0.158,551,551
      c3_f_100KB.txt_SIZE_r2,3,"SIZE
f_100KB.txt",20,20,20,0,0,0.304,5.331,0.222,0.142,551,551
      c3_f_100KB.txt_SIZE_r3,1,"SIZE
f_100KB.txt",20,20,20,0,0,0.403,5.764,0.234,0.148,551,551
      c3_f_100KB.txt_SIZE_r3,2,"SIZE
f_100KB.txt",20,20,20,0,0,0.343,5.487,0.226,0.144,551,551
      c3_f_100KB.txt_SIZE_r3,3,"SIZE
f_100KB.txt",20,20,20,0,0,0.340,5.103,0.209,0.134,551,551
      c3_f_100KB.txt_COUNT_LINES_r1,1,"COUNT_LINES
f_100KB.txt",20,20,20,0,0,0.345,9.862,0.443,0.306,691,511
      c3_f_100KB.txt_COUNT_LINES_r1,2,"COUNT_LINES
f_100KB.txt",20,20,20,0,0,0.310,10.042,0.456,0.347,691,511
      c3_f_100KB.txt_COUNT_LINES_r1,3,"COUNT_LINES
f_100KB.txt",20,20,20,0,0,0.366,8.476,0.372,0.279,691,511
      c3_f_100KB.txt_COUNT_LINES_r2,1,"COUNT_LINES
f_100KB.txt",20,20,20,0,0,0.367,8.526,0.372,0.272,691,511
      c3_f_100KB.txt_COUNT_LINES_r2,2,"COUNT_LINES
f_100KB.txt",20,20,20,0,0,0.354,8.913,0.395,0.294,691,511
      c3_f_100KB.txt_COUNT_LINES_r2,3,"COUNT_LINES
f_100KB.txt",20,20,20,0,0,0.356,8.367,0.368,0.264,691,511
      c3_f_100KB.txt_COUNT_LINES_r3,1,"COUNT_LINES
f_100KB.txt",20,20,20,0,0,0.414,9.136,0.398,0.290,691,511
      c3_f_100KB.txt_COUNT_LINES_r3,2,"COUNT_LINES
f_100KB.txt",20,20,20,0,0,0.418,9.460,0.416,0.312,691,511
      c3_f_100KB.txt_COUNT_LINES_r3,3,"COUNT_LINES
f_100KB.txt",20,20,20,0,0,0.340,8.273,0.363,0.262,691,511
      c3_f_100KB.txt_READ_FIRST_r1,1,"READ_FIRST                  f_100KB.txt
100",20,20,20,0,0,0.344,11.217,0.510,0.384,751,94191
      c3_f_100KB.txt_READ_FIRST_r1,2,"READ_FIRST                  f_100KB.txt
100",20,20,20,0,0,0.400,10.924,0.489,0.376,751,94191
      c3_f_100KB.txt_READ_FIRST_r1,3,"READ_FIRST                  f_100KB.txt
100",20,20,20,0,0,0.345,9.895,0.446,0.330,751,94191
      c3_f_100KB.txt_READ_FIRST_r2,1,"READ_FIRST                  f_100KB.txt
100",20,20,20,0,0,0.398,12.179,0.550,0.409,751,94191
      c3_f_100KB.txt_READ_FIRST_r2,2,"READ_FIRST                  f_100KB.txt
100",20,20,20,0,0,0.372,11.673,0.527,0.398,751,94191
      c3_f_100KB.txt_READ_FIRST_r2,3,"READ_FIRST                  f_100KB.txt
100",20,20,20,0,0,0.443,10.538,0.468,0.348,751,94191
      c3_f_100KB.txt_READ_FIRST_r3,1,"READ_FIRST                  f_100KB.txt
100",20,20,20,0,0,0.531,12.469,0.547,0.407,751,94191
      c3_f_100KB.txt_READ_FIRST_r3,2,"READ_FIRST                  f_100KB.txt
100",20,20,20,0,0,0.322,12.051,0.545,0.416,751,94191
      c3_f_100KB.txt_READ_FIRST_r3,3,"READ_FIRST                  f_100KB.txt
100",20,20,20,0,0,0.513,10.598,0.469,0.347,751,94191
      c3_f_10MB.txt_SIZE_r1,1,"SIZE
f_10MB.txt",20,20,20,0,0,0.394,7.198,0.306,0.187,531,591
```

```
      c3_f_10MB.txt_SIZE_r1,2,"SIZE
f_10MB.txt",20,20,20,0,0,0.415,6.965,0.283,0.179,531,591
      c3_f_10MB.txt_SIZE_r1,3,"SIZE
f_10MB.txt",20,20,20,0,0,0.489,6.542,0.262,0.165,531,591
      c3_f_10MB.txt_SIZE_r2,1,"SIZE
f_10MB.txt",20,20,20,0,0,0.472,8.394,0.353,0.215,531,591
      c3_f_10MB.txt_SIZE_r2,2,"SIZE
f_10MB.txt",20,20,20,0,0,0.418,8.297,0.347,0.229,531,591
      c3_f_10MB.txt_SIZE_r2,3,"SIZE
f_10MB.txt",20,20,20,0,0,0.488,6.235,0.244,0.141,531,591
      c3_f_10MB.txt_SIZE_r3,1,"SIZE
f_10MB.txt",20,20,20,0,0,0.353,6.776,0.286,0.176,531,591
      c3_f_10MB.txt_SIZE_r3,2,"SIZE
f_10MB.txt",20,20,20,0,0,0.352,7.086,0.305,0.201,531,591
      c3_f_10MB.txt_SIZE_r3,3,"SIZE
f_10MB.txt",20,20,20,0,0,0.303,5.351,0.222,0.139,531,591
      c3_f_10MB.txt_COUNT_LINES_r1,1,"COUNT_LINES
f_10MB.txt",20,20,20,0,0,0.493,262.265,13.045,12.812,671,590
      c3_f_10MB.txt_COUNT_LINES_r1,3,"COUNT_LINES
f_10MB.txt",20,20,20,0,0,0.383,264.047,13.145,12.889,671,591
      c3_f_10MB.txt_COUNT_LINES_r1,2,"COUNT_LINES
f_10MB.txt",20,20,20,0,0,0.485,271.324,13.501,13.272,671,591
      c3_f_10MB.txt_COUNT_LINES_r2,1,"COUNT_LINES
f_10MB.txt",20,20,20,0,0,0.341,244.556,12.173,11.957,671,591
      c3_f_10MB.txt_COUNT_LINES_r2,3,"COUNT_LINES
f_10MB.txt",20,20,20,0,0,0.344,241.641,12.029,11.801,671,590
      c3_f_10MB.txt_COUNT_LINES_r3,2,"COUNT_LINES
f_10MB.txt",20,20,20,0,0,0.331,249.908,12.442,12.246,671,591
      c3_f_10MB.txt_COUNT_LINES_r3,3,"COUNT_LINES
f_10MB.txt",20,20,20,0,0,0.321,253.606,12.628,12.407,671,590
      c3_f_10MB.txt_READ_FIRST_r1,1,"READ_FIRST                    f_10MB.txt
100",20,20,20,0,0,0.393,10.647,0.477,0.360,731,103111
      c3_f_10MB.txt_READ_FIRST_r1,2,"READ_FIRST                    f_10MB.txt
100",20,20,20,0,0,0.418,12.206,0.551,0.419,731,103112
      c3_f_10MB.txt_READ_FIRST_r1,3,"READ_FIRST                    f_10MB.txt
100",20,20,20,0,0,0.403,11.329,0.509,0.376,731,103111
      c3_f_10MB.txt_READ_FIRST_r2,1,"READ_FIRST                    f_10MB.txt
100",20,20,20,0,0,0.339,9.740,0.440,0.329,731,103111
      c3_f_10MB.txt_READ_FIRST_r2,2,"READ_FIRST                    f_10MB.txt
100",20,20,20,0,0,0.300,9.511,0.426,0.332,731,103111
      c3_f_10MB.txt_READ_FIRST_r2,3,"READ_FIRST                    f_10MB.txt
100",20,20,20,0,0,0.407,9.425,0.417,0.318,731,103111
      c3_f_10MB.txt_READ_FIRST_r3,1,"READ_FIRST                    f_10MB.txt
100",20,20,20,0,0,0.379,11.459,0.517,0.382,731,103111
      c3_f_10MB.txt_READ_FIRST_r3,2,"READ_FIRST                    f_10MB.txt
100",20,20,20,0,0,0.369,11.605,0.526,0.417,731,103111
      c3_f_10MB.txt_READ_FIRST_r3,3,"READ_FIRST                    f_10MB.txt
100",20,20,20,0,0,0.361,10.212,0.459,0.348,731,103111
      c3_f_50MB.txt_SIZE_r1,1,"SIZE
f_50MB.txt",20,20,20,0,0,0.398,6.471,0.269,0.165,531,591
      c3_f_50MB.txt_SIZE_r1,2,"SIZE
f_50MB.txt",20,20,20,0,0,0.363,6.348,0.264,0.166,531,591
      c3_f_50MB.txt_SIZE_r1,3,"SIZE
f_50MB.txt",20,20,20,0,0,0.426,5.240,0.210,0.132,531,591
      c3_f_50MB.txt_SIZE_r2,1,"SIZE
f_50MB.txt",20,20,20,0,0,0.391,6.563,0.277,0.170,531,591
      c3_f_50MB.txt_SIZE_r2,2,"SIZE
f_50MB.txt",20,20,20,0,0,0.337,6.654,0.280,0.172,531,591
      c3_f_50MB.txt_SIZE_r2,3,"SIZE
f_50MB.txt",20,20,20,0,0,0.305,6.019,0.251,0.155,531,591
      c3_f_50MB.txt_SIZE_r3,1,"SIZE
f_50MB.txt",20,20,20,0,0,0.376,6.183,0.257,0.157,531,591
      c3_f_50MB.txt_SIZE_r3,2,"SIZE
f_50MB.txt",20,20,20,0,0,0.358,6.774,0.286,0.180,531,591
```

```
      c3_f_50MB.txt_SIZE_r3,3,"SIZE
f_50MB.txt",20,20,20,0,0,0.362,5.579,0.230,0.146,531,591
      c3_f_50MB.txt_COUNT_LINES_r1,1,"COUNT_LINES
f_50MB.txt",20,20,20,0,0,0.364,1315.987,65.736,65.445,671,611
      c3_f_50MB.txt_COUNT_LINES_r1,2,"COUNT_LINES
f_50MB.txt",20,20,20,0,0,0.414,1325.524,66.212,65.971,671,611
      c3_f_50MB.txt_COUNT_LINES_r2,1,"COUNT_LINES
f_50MB.txt",20,20,20,0,0,0.351,1343.858,67.132,66.844,671,611
      c3_f_50MB.txt_COUNT_LINES_r2,3,"COUNT_LINES
f_50MB.txt",20,20,20,0,0,0.315,1359.928,67.941,67.689,671,611
      c3_f_50MB.txt_COUNT_LINES_r2,2,"COUNT_LINES
f_50MB.txt",20,20,20,0,0,0.392,1369.229,68.398,68.169,671,611
      c3_f_50MB.txt_COUNT_LINES_r3,1,"COUNT_LINES
f_50MB.txt",20,20,20,0,0,0.355,1401.204,70.000,69.702,671,611
      c3_f_50MB.txt_COUNT_LINES_r3,3,"COUNT_LINES
f_50MB.txt",20,20,20,0,0,0.623,1434.977,71.670,71.387,671,612
      c3_f_50MB.txt_COUNT_LINES_r3,2,"COUNT_LINES
f_50MB.txt",20,20,20,0,0,0.419,1443.168,72.093,71.828,671,612
      c3_f_50MB.txt_READ_FIRST_r1,1,"READ_FIRST                    f_50MB.txt
100",20,20,20,0,0,0.362,8.988,0.398,0.302,731,100551
      c3_f_50MB.txt_READ_FIRST_r1,2,"READ_FIRST                    f_50MB.txt
100",20,20,20,0,0,0.285,9.530,0.434,0.342,731,100551
      c3_f_50MB.txt_READ_FIRST_r1,3,"READ_FIRST                    f_50MB.txt
100",20,20,20,0,0,0.390,8.554,0.372,0.280,731,100551
      c3_f_50MB.txt_READ_FIRST_r2,1,"READ_FIRST                    f_50MB.txt
100",20,20,20,0,0,0.325,11.171,0.510,0.385,731,100551
      c3_f_50MB.txt_READ_FIRST_r2,2,"READ_FIRST                    f_50MB.txt
100",20,20,20,0,0,0.334,11.862,0.538,0.400,731,100551
      c3_f_50MB.txt_READ_FIRST_r2,3,"READ_FIRST                    f_50MB.txt
100",20,20,20,0,0,0.340,11.590,0.525,0.404,731,100551
      c3_f_50MB.txt_READ_FIRST_r3,1,"READ_FIRST                    f_50MB.txt
100",20,20,20,0,0,0.515,11.798,0.527,0.389,731,100551
      c3_f_50MB.txt_READ_FIRST_r3,2,"READ_FIRST                    f_50MB.txt
100",20,20,20,0,0,0.402,11.739,0.529,0.410,731,100551
      c3_f_50MB.txt_READ_FIRST_r3,3,"READ_FIRST                    f_50MB.txt
100",20,20,20,0,0,0.410,10.305,0.457,0.349,731,100551
```